UD2.2. Comprensión de listas

Introducción al uso de Python

Sintaxis

La comprensión de listas es una forma concisa y visual de crear o filtrar listas a partir de otras colecciones de datos, en una sola línea.

La expresión base es la siguiente:

[expresión for elemento in iterable if condicion]

Ejemplo 1

```
numeros = range(1,11)
pares = [n for n in numeros if n % 2 == 0]
print(pares)
```

En este ejemplo, damos **n** como nombre a cada uno de los elementos de la lista **numeros**, que contiene los valores enteros del 1 al 10.

Podríamos darle cualquier nombre a esta variable, pero en la comprensión de listas se recomienda usar una única letra que sea significativa.

Ejemplo 2

```
notas = [4, 8, 3, 9, 5]


resultados = ["Aprobado" if nota >= 5 else "Suspenso" for nota in notas]
print(resultados)
```

Ejemplo 3

Podemos iterar también sobre una colección que sea una lista de diccionarios.

```
nueva_lista = [p for p in pokemons if p["peso_kg"] > 10]
print(nueva_lista)
```

Ejemplo 4

No es necesario iterar sobre toda la colección, podemos iterar solo sobre una de las claves.

```
nombres_pokemons = [p["nombre"] for p in pokemons]
print(nombres_pokemons)
```

Ejemplo 5

También podemos crear una nueva lista con varias de las claves de la colección.

```
nombres_pokemons = [(p["nombre"], p["peso_kg"]) for p in pokemons]
print(nombres_pokemons)
```

Ejemplo 6

Igualmente, podemos crear una lista de diccionarios con pares clave-valor.

```
pokemons2 = [{
    "nombre" : p["nombre"],
    "peso_kg" : p["peso_kg"]}
    for p in pokemons]
print(pokemons2)
```

El operador IN

El operador **in** comprueba si un elemento se encuentra dentro de una secuencia (lista, tupla, diccionario o cadena de caracteres).

Es equivalente a recorrer la secuencia con un bucle para comprobar si existe el valor.

El operador opuesto es **not in**, que comprueba si un elemento no se encuentra dentro de una secuencia.

El operador IN

```
numeros = [1,2,6,7,10]


if 7 in numeros:
    print(El 7 está)
```

El operador *

El operador *, puesto como operador unario al comienzo de una compresión de listas, permite “desempaquetar” cada uno de los elementos sobre los que se itera.

El operador *

```
listas = [[1, 2], [3, 4], [5, 6]]


# Con el asterisco al principio de la expresión:
resultado = [*sublista for sublista in listas]


print(resultado)
# Salida: [1, 2, 3, 4, 5, 6]
```

El operador *

Una de sus principales utilidades es imprimir una lista